Plano de aula e material de apoio aula ao vivo

Aula 01

Paradigmas e modelagem orientada a objetos

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Bem-vindos!

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Apresentação do Tutor

SOBRE VOCÊ

Apresente-se para a turma, compartilhando sua trajetória, motivações e área de atuação.

EXPERIÊNCIA PROFISSIONAL

Destaque suas principais vivências de mercado, formação acadêmica e especialidades técnicas.



Objetivo da Aula e Capacidade

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

ORIENTAÇÕES PEDAGÓGICAS

OBJETIVO DA AULA

Compreender o que é um paradigma de programação, diferenciar o paradigma estruturado do orientado a objetos e preparar a modelagem do cadastro de clientes da Electronic com o diagrama de classes UML.

CAPACIDADES

Básicas: reconhecer a definição de paradigma de programação; reconhecer os tipos e classificações das linguagens conforme os paradigmas; aplicar as características da POO no desenvolvimento de aplicativos.

Socioemocional: constatar a iniciativa como requisito fundamental para uma postura inovadora e aberta a novas experiências e aprendizados.

Conhecimentos Relacionados

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

ORIENTAÇÕES PEDAGÓGICAS

POR QUE ESTES CONHECIMENTOS?

Eles sustentam a modelagem do cadastro de pessoas físicas e jurídicas do aplicativo da Electronic e preparam a implementação em JavaScript, que será vista na Aula 2.

CONHECIMENTOS

- Paradigmas de programação: definição
- Tipos de paradigmas: estruturado e orientado a objetos
- Classes, objetos, atributos e métodos
- Herança e abstração
- Encapsulamento e polimorfismo
- Interfaces
- Diagrama de classes UML

Recursos e Ambientes Pedagógicos

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

ORIENTAÇÕES PEDAGÓGICAS

01

AVA

Fórum de Discussão, entrega das atividades e fórum de dúvidas.

02

Draw.io

Ferramenta gratuita para criar o diagrama de classes UML (app.diagrams.net).

03

Enquetes

Perguntas rápidas durante a aula para checar a compreensão da turma.

04

Material digital

Desafios 1 e 2 do material da unidade, para estudar antes e depois da aula.

Roteiro de Estudos e Avaliações

ORIENTAÇÕES PEDAGÓGICAS

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Semana 1	
Estudar	**Livro Didático** Desafio 1 – Paradigmas de programação e orientação a objetos, pág. 5 a 29. **Material Online** Aula 1: Paradigmas de programação e orientação a objetos (leitura de todas as telas) Vídeo de apoio: POO
Participar	Aula ao vivo 1: Paradigmas e modelagem orientada a objetos Fórum de dúvidas
Responder	**Fórum de Discussão** **Avaliação Online 1**

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Agenda da aula

Momento	Tempo	Conteúdo
1. Paradigmas	30 min	Situação-problema da Electronic e paradigmas estruturado e orientado a objetos
2. Pilares da POO	40 min	Classe, objeto, abstração, encapsulamento, herança, polimorfismo e interfaces
3. Diagrama de classes	40 min	Notação UML e demonstração no Draw.io
4. Atividades	10 min	Lançamento do Fórum e da Atividade 2 e tira-dúvidas

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

O desafio da Electronic

- A Appfull desenvolve o aplicativo de e-commerce da Electronic, empresa de eletrônicos de atacado e varejo.
- O sistema deve armazenar cadastros de pessoas físicas e jurídicas, sempre com endereço.
  - Pessoa física: nome, CPF e data de nascimento
  - Pessoa jurídica: nome, CNPJ e razão social
- Valores não permitidos digitados pelos usuários precisam ser tratados.
- Desafio 1: reconhecer os paradigmas. Desafio 2: aplicar a POO no aplicativo.

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

O que é um paradigma?

- Paradigma é uma abordagem para resolver problemas, com técnicas e ferramentas que podem ser aplicadas a diferentes linguagens.
- Uma linguagem pode aplicar mais de um paradigma; o importante é escolher o mais adequado ao projeto.
- As linguagens também são classificadas como compiladas ou interpretadas e de alto ou baixo nível.
- Os paradigmas mais usados: programação estruturada e programação orientada a objetos.

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Estruturado x POO

Característica	Programação estruturada	Programação orientada a objetos
Organização	Top-down: o problema é dividido em partes cada vez menores	Sistema formado por objetos que interagem entre si
Dados e funções	Separados	Reunidos no objeto: atributos e métodos
Mudanças	Funções compartilham os dados; mudar exige cuidado	Partes podem ser alteradas ou substituídas com menos risco de erros
Exemplos	C, Pascal, Cobol, Fortran	Java, JavaScript, C#, Python, Dart

Um olhar prático

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

EXEMPLO PRÁTICO

PESSOA X: ESTRUTURADO

Os dados (triângulo, quadrado e círculo) ficam separados das funções (mover, rotacionar e mudar tamanho). Qualquer função pode ser aplicada a qualquer forma.

Para pensar: e se surgir uma nova forma, como o pentágono?

PESSOA Y: ORIENTADO A OBJETOS

Cada forma é um objeto com os métodos mover, rotacionar e mudar tamanho. Dados e comportamento ficam juntos, no próprio objeto.

Para pensar: o que um novo objeto precisa ter para ser movido e rotacionado?

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Classe, objeto e construtor

- Classe é o molde: define os atributos (dados) e os métodos (comportamentos) de um conjunto de objetos.
- Objeto é uma instância da classe, com identidade e valores próprios de atributos.
  - Ex.: a classe Carro (cor, modelo, andar(), frear()) gera CarroDaCris e CarroDaClaudia.
- Construtor é o método executado quando o objeto é criado; ele não retorna valor.
- Modificadores de acesso definem a visibilidade: público, protegido e privado.

Os pilares da POO

ORIENTAÇÃO A OBJETOS

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

01

Abstração

Focar no essencial: expor as operações e esconder os detalhes de implementação.

02

Encapsulamento

Atributos protegidos; o acesso acontece pelos métodos do próprio objeto.

03

Herança

Classes derivadas reaproveitam atributos e métodos da classe base.

04

Polimorfismo

Mesma assinatura de método, com comportamento específico em cada classe derivada.

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Abstração

- Abstrair é representar algo do mundo real apenas com as características que importam para o sistema.
- Ex.: num sistema bancário, a Conta precisa de titular e saldo; a cor dos olhos do cliente não interessa.
- Quem usa a classe conhece as operações (depositar, sacar), não a forma como são implementadas.
- Em Java, classes abstratas e interfaces são as ferramentas da abstração.

EXEMPLO EM JAVA

```
public interface Conta {
    void depositar(double valor);
    void sacar(double valor);
    double getSaldo();
}

// quem usa sabe O QUE a conta faz,
// não COMO ela faz
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Encapsulamento

- Os atributos ficam privados: nenhum código externo altera o estado do objeto diretamente.
- O acesso passa por métodos públicos (getters, setters e operações), que validam os dados.
- Ex.: o saldo só muda por depositar() ou sacar(), que recusam valores inválidos.
- A implementação interna pode mudar sem quebrar o código de quem usa a classe.

EXEMPLO EM JAVA

```
public class Conta {
    private double saldo;

    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
    }

    public double getSaldo() {
        return saldo;
    }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Herança

- A classe derivada (filha) recebe atributos e métodos da classe base (mãe) e pode acrescentar os seus.
- Representa a relação "é um": Poupança é uma Conta.
- Evita duplicar código: o que é comum a todas as contas fica na classe base.
- Em Java, usa-se extends, e cada classe herda de uma única classe base.

EXEMPLO EM JAVA

```
public class Poupanca extends Conta {
    private Date aniversario;

    public void renderJuros(double taxa) {
        // depositar() e getSaldo()
        // vêm herdados de Conta
        depositar(getSaldo() * taxa);
    }
}
```

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Polimorfismo

- "Muitas formas": o mesmo método se comporta de modo diferente em cada classe derivada.
- Na sobrescrita (@Override), a classe filha redefine um método herdado com a mesma assinatura.
- O código trata todos os objetos como Conta; em tempo de execução, o Java escolhe a versão certa.
- Ex.: sacar() na Corrente pode usar o limite; na Poupança, só o saldo.

EXEMPLO EM JAVA

```
public class Corrente extends Conta {
    @Override
    public void sacar(double valor) {
        // regra própria: usa o limite
    }
}

Conta c = new Corrente();
c.sacar(100);
// executa o sacar() de Corrente
```

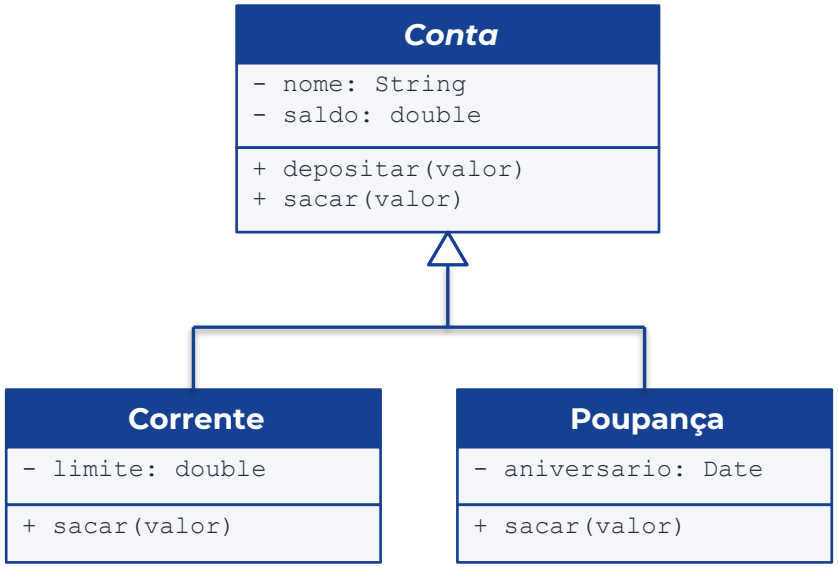
CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Abstratas e interfaces

- Classe abstrata serve de base para outras e não gera instâncias; pode ter métodos abstratos (sem implementação) e concretos.
- As classes derivadas concretas devem implementar os métodos abstratos.
- Interface é um contrato: define o que a classe deve oferecer, sem implementar.
- Uma classe pode implementar várias interfaces, o que ajuda quando não há herança múltipla, como no Java.

Diagrama de classes UML

- A classe é um retângulo com três partes: nome, atributos e métodos.
- Visibilidade: + público, - privado e # protegido.
- A seta com triângulo vazado liga a classe derivada à classe base (herança).
- Classe abstrata: nome em itálico.
- Ferramenta gratuita: Draw.io (app.diagrams.net).



CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Atividades da unidade

Atividade	Peso	Forma de entrega
1. Fórum de Discussão	10%	Mensagens enviadas no Fórum
2. Modelagem do sistema (UML)	40%	Arquivo do diagrama publicado no AVA
3. Implementação em JavaScript	50%	Link do GitHub compartilhado no AVA
Total	100%	40h de carga horária online

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Atividade: Fórum

- Tema: paradigmas de programação estruturado e orientado a objetos.
- Pesquise e compartilhe ao menos dois exemplos de sistemas e aplicações, indicando o paradigma mais adequado a cada um.
- Comente ao menos uma postagem dos colegas, com cordialidade e respeito.
- Critérios: participar no prazo, compartilhar os exemplos e interagir com a turma (peso de 10%).

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Atividade: Modelagem UML

- Crie o diagrama de classes do cadastro de clientes da Electronic, aplicando os princípios da orientação a objetos.
  - Pessoa física: nome, CPF, data de nascimento e endereço
  - Pessoa jurídica: nome, CNPJ, razão social e endereço
- Perguntas-guia: quais são as classes? O que é comum às duas? O que é específico de cada uma?
- Salve o arquivo do Draw.io e publique no AVA, dentro do prazo (peso de 40%).

CTMSENAI
CENTRAL DE TUTORIA E MONITORIA

Encerramento

- Paradigmas: a programação estruturada separa dados e funções; a POO reúne atributos e métodos em objetos.
- Pilares: abstração, encapsulamento, herança e polimorfismo.
- Diagrama de classes UML: nome, atributos, métodos e relações entre classes.
- Próximos passos: participe do Fórum, desenvolva a Atividade e traga dúvidas para a Aula 2.

Obrigado!

CTMSENAI

CENTRAL DE TUTORIA E MONITORIA